

Solo-Git Phase 2: Comprehensive Gap Analysis

Generated: November 3, 2025
Branch: phase-2-audit-refactor
Analysis Date: 2025-11-03

Executive Summary

This document presents a comprehensive gap analysis of the Solo-Git codebase, identifying:

- **Undocumented features** - Code that exists but isn't documented
- **Documented-but-missing features** - Documentation without implementation
- **Inconsistent behaviors** - Similar operations with different patterns
- **Dead code** - Unused or commented-out code
- **Duplicated logic** - Functions/patterns appearing in multiple places
- **Test coverage gaps** - Modules without adequate tests
- **Code quality issues** - Large functions, missing type hints, etc.

Key Findings

Category	Count	Priority	Status
Undocumented public functions	86	Medium	⚠ Needs documentation
Large functions (>100 lines)	14	High	🔴 Refactor candidates
Duplicate function names	88	Medium	⚠ Review for consolidation
Code markers (TODO/FIXME)	5	Low	✅ Minimal technical debt
Modules without dedicated tests	24	High	🔴 Critical gap
Functions without type hints	65	Medium	⚠ Type safety gap
Commented-out code blocks	1	Low	✅ Minimal dead code

Overall Health: 🟡 Good with areas needing improvement

1. Undocumented Features

1.1 Summary

Total: 86 undocumented public functions and classes

Priority: Medium (impacts maintainability)

1.2 Detailed Breakdown by Module

API Client (`sologit/api/client.py`)

- `chat()` - Line 394
- `stream_chat()` - Line 435
- `safe_int()` - Line 313
- `update_from()` - Line 319

Recommendation: Add docstrings following Google style:

```
def chat(self, messages: List[Dict], model: str = None) -> ChatResponse:
    """Send a chat completion request.

    Args:
        messages: List of message dictionaries with 'role' and 'content'
        model: Optional model override

    Returns:
        ChatResponse object with completion and metadata

    Raises:
        AbacusAPIError: If API request fails
    """
```

CLI Commands (`sologit/cli/`)

- `commands.py::progress_callback()` - Line 1177
- `commands.py::handle_output()` - Line 775
- `commands.py::handle_complete()` - Line 783
- `ci_commands.py::progress_callback()` - Line 48
- `enhanced_commands.py::git_engine()` - Line 33
- `enhanced_commands.py::patch_engine()` - Line 39
- `enhanced_commands.py::test_orchestrator()` - Line 45

Recommendation: Add docstrings to all CLI helper functions and callbacks.

State Management (`sologit/state/`)

- `manager.py::read_global_state()` - Line 220
- `manager.py::write_global_state()` - Line 227
- `manager.py::read_repository()` - Line 232
- `manager.py::write_repository()` - Line 237
- `manager.py::list_repositories()` - Line 242

Recommendation: Document all StateBackend interface methods.

Core Models (`sologit/core/workpad.py`)

- `Snapshot.to_dict()` - Line 25

- `Snapshot.from_dict()` - Line 33

Recommendation: Add serialization documentation.

1.3 Action Items

1. Immediate (High Priority):

- Document all public API methods in `api/client.py`
- Document StateBackend interface methods

2. Short-term (Medium Priority):

- Add docstrings to CLI callbacks and helpers
- Document serialization methods in core models

3. Long-term (Low Priority):

- Add examples to complex functions
- Create API reference documentation

2. Code Markers (Technical Debt)

2.1 Summary

Total: 5 markers found

Priority: Low (minimal technical debt)

2.2 Detailed Breakdown

All markers are in `sologit/orchestration/code_generator.py` :

Line	Marker	Context
308	TODO	<code>patch += "+# TODO: Implement actual changes\n"</code>
331	TODO	<code>'# TODO: Implement this module'</code>
349	TODO	<code>patch += f"+# TODO: Implement changes for: {plan.title}\n"</code>
366	TODO	<code># Create a simple TODO patch</code>
370	TODO	<code>diff += f"+# TODO: {plan.title}\n"</code>

2.3 Analysis

These TODOs are **intentional placeholder text** being generated in patches, not actual code debt.

They are:

- Part of the patch generation logic

- Used as placeholders when AI generates incomplete patches
- Working as designed

Status:  **No action required** - These are features, not debt.

3. Code Duplication

3.1 Summary

Total: 88 function names appearing in multiple files

Priority: Medium (some consolidation opportunities)

3.2 Critical Duplication (3+ files)

`abort_with_error` - 4 files

- `cli/main.py`
- `cli/config_commands.py`
- `cli/commands.py`
- `cli/integrated_commands.py`

Issue: Error handling utility duplicated across CLI modules.

Recommendation: Consolidate into `cli/utils.py` :

```
# cli/utils.py
def abort_with_error(message: str, code: int = 1) -> None:
    """Print error message and exit.

    Args:
        message: Error message to display
        code: Exit code (default: 1)
    """
    console = Console()
    console.print(f"[red]Error:[/red] {message}")
    raise click.Abort()
```

`action_help` , `action_refresh` - 3 files each

- `ui/enhanced_tui.py`
- `ui/tui_app.py`
- `ui/heaven_tui.py`

Issue: TUI action handlers duplicated across multiple TUI implementations.

Recommendation: Create `ui/tui_actions.py` base class:

```
class BaseTUIActions:
    """Base class for common TUI actions."""

    def action_help(self) -> None:
        """Show help overlay."""
        pass

    def action_refresh(self) -> None:
        """Refresh current view."""
        pass
```

apply_patch - 3 files

- state/git_sync.py
- engines/git_engine.py
- engines/patch_engine.py

Issue: Patch application logic duplicated.

Analysis:

- engines/patch_engine.py::apply_patch() - Core implementation ✓
- engines/git_engine.py::apply_patch() - Wrapper with Git context ✓
- state/git_sync.py::apply_patch() - State sync wrapper ✓

Status: ✓ **No consolidation needed** - Different layers, appropriate separation.

create_workpad - 3 files

- state/manager.py
- state/git_sync.py
- engines/git_engine.py

Status: ✓ **No consolidation needed** - Layered architecture (Engine → Sync → State).

compose - 8 files (Textual UI widgets)

All in ui/ directory - Textual framework method overrides.

Status: ✓ **No consolidation needed** - Framework requirement.

3.3 Action Items

1. **High Priority:** Consolidate abort_with_error into cli/utils.py
2. **Medium Priority:** Create ui/tui_actions.py base class for TUI actions
3. **Low Priority:** Document the intentional duplication for clarity

4. Large Functions (Refactoring Candidates)

4.1 Summary

Total: 14 functions over 100 lines

Priority: High (impacts maintainability and testability)

4.2 Top Refactoring Candidates

1. `cli/commands.py::execute_pair_loop` - 283 lines

Location: Line 1292

Issue: Massive function handling entire AI pairing workflow

Impact: Hard to test, hard to maintain, hard to extend

Recommendation: Break into smaller functions:

```
def execute_pair_loop(prompt: str, context: Dict) -> PairResult:
    """Execute AI pair programming loop."""
    # 1. Parse and validate prompt
    task = _parse_pair_prompt(prompt)

    # 2. Plan changes
    plan = _plan_changes(task, context)

    # 3. Generate patches
    patches = _generate_patches(plan)

    # 4. Apply patches
    results = _apply_patches(patches)

    # 5. Run tests
    test_results = _run_pair_tests(results)

    # 6. Handle results
    return _handle_pair_results(test_results)
```

Estimated Refactor Time: 3-4 hours

Test Improvement: ~50% increase in testability

2. `cli/commands.py::test_run` - 252 lines

Location: Line 719

Issue: Handles test execution, progress display, and result formatting

Recommendation: Extract into separate concerns:

- `_execute_tests()` - Test execution logic
- `_display_test_progress()` - Progress bar/spinner
- `_format_test_results()` - Result formatting
- `_handle_test_failures()` - Failure handling

Estimated Refactor Time: 2-3 hours

3. `cli/commands.py::generate_commit_message` - 201 lines

Location: Line 1585

Issue: AI commit message generation + interactive editing + error handling

Recommendation: Extract:

- `_build_commit_context()` - Gather diff and context
- `_generate_message_with_ai()` - AI generation

- `_interactive_edit()` - Editor interaction
- `_finalize_commit()` - Apply message and commit

Estimated Refactor Time: 2 hours

4. `workflows/auto_merge.py::execute` - 183 lines ●

Location: Line 92

Issue: Entire auto-merge workflow in one function

Current Structure: One large `execute()` method with inline steps.

Recommended Structure:

```
class AutoMergeWorkflow:
    def execute(self) -> AutoMergeResult:
        """Execute auto-merge workflow with proper separation."""
        # Step 1
        tests = self._run_tests()
        if not tests.passed:
            return self._handle_test_failure(tests)

        # Step 2
        gate = self._check_promotion_gate()
        if not gate.approved:
            return self._handle_gate_rejection(gate)

        # Step 3
        promotion = self._promote_workpad()
        if not promotion.success:
            return self._handle_promotion_failure(promotion)

        # Step 4
        ci = self._run_ci_tests()
        if not ci.passed:
            return self._handle_ci_failure(ci)

        return self._success_result()
```

Benefits:

- Each step independently testable
- Clear error handling per step
- Easy to add/remove steps
- Better logging and progress tracking

Estimated Refactor Time: 3-4 hours

5. `orchestration/code_generator.py::generate_patch` - 141 lines ●

Location: Line 89

Recommendation: Extract:

- `_build_generation_prompt()` - Prompt construction
- `_call_ai_model()` - API call

- `_parse_ai_response()` - Response parsing
- `_validate_patch()` - Patch validation

Estimated Refactor Time: 2 hours

4.3 Additional Refactoring Candidates

Function	Lines	Priority	Estimated Time
<code>cli/main.py::_run_interactive_shell</code>	130	Medium	1-2 hours
<code>workflows/auto_merge.py::_run_ci_pipeline</code>	129	High	2 hours
<code>orchestration/ai_orchestrator.py::plan</code>	128	High	2 hours
<code>ui/heaven_tui.py::_setup_commands</code>	121	Low	1 hour
<code>workflows/promotion_gate.py::evaluate</code>	118	Medium	2 hours
<code>orchestration/ai_orchestrator.py::diagnose_failure</code>	118	Medium	2 hours
<code>workflows/ci_orchestrator.py::run_smoke_tests</code>	111	Medium	1-2 hours
<code>orchestration/ai_orchestrator.py::generate_patch</code>	111	High	2 hours
<code>cli/commands.py::pad_auto_merge</code>	101	Medium	1-2 hours

4.4 Total Refactoring Effort Estimate

- **High Priority** (6 functions): ~14-18 hours
- **Medium Priority** (6 functions): ~8-12 hours

- **Low Priority** (2 functions): ~2 hours
- **Total:** ~24-32 hours (~3-4 working days)

5. Inconsistent Naming Patterns

5.1 Summary

Similar operations use different verb prefixes, reducing code readability.

5.2 Verb Prefix Analysis

Prefix	Count	Common Usage
get_*	71	Most common - retrieving data
create_*	15	Object creation
read_*	14	File/state reading
fetch_*	5	(Should consolidate with get)
retrieve_*	3	(Should consolidate with get)
load_*	8	(Overlap with read)
make_*	4	(Overlap with create)
build_*	6	(Overlap with create)

5.3 Recommended Naming Conventions

Adopt clear, consistent patterns:

Data Retrieval

-  **Use:** `get_*` for in-memory object retrieval
-  **Use:** `fetch_*` for external API calls
-  **Use:** `read_*` for file/disk operations
-  **Avoid:** `retrieve_*`, `obtain_*`, `acquire_*`

Object Creation

-  **Use:** `create_*` for business objects (workpads, repositories)
-  **Use:** `build_*` for constructed/assembled objects (prompts, configs)
-  **Use:** `make_*` for simple factory functions
-  **Avoid:** mixing patterns

Examples

Good (Consistent):

```
# Data retrieval
def get_workpad(workpad_id: str) -> Workpad: ...      # In-memory
def fetch_ai_response(prompt: str) -> str: ...         # External API
def read_config_file(path: Path) -> Dict: ...         # File I/O

# Object creation
def create_workpad(title: str) -> Workpad: ...        # Business object
def build_prompt(context: Dict) -> str: ...           # Assembled string
def make_temp_dir() -> Path: ...                     # Simple factory
```

Bad (Inconsistent):

```
def retrieve_workpad() ...      # Should be get_workpad()
def obtain_config() ...        # Should be read_config()
def construct_prompt() ...     # Should be build_prompt()
```

5.4 Refactoring Impact

Scope: ~25 function renames across codebase

Breaking Changes: Internal API only (no public CLI changes)

Estimated Time: 2-3 hours (find & replace with tests)

5.5 Action Items

1. **Phase 1:** Audit all function names and document current patterns
2. **Phase 2:** Create naming convention guide
3. **Phase 3:** Refactor in small batches (5-10 functions at a time)
4. **Phase 4:** Update tests and documentation

6. Error Handling Patterns

6.1 Summary

Multiple error handling patterns in use, reducing consistency.

6.2 Current Patterns

Pattern	Occurrences	Usage
<code>return None</code>	30	Error indication
<code>raise RuntimeError</code>	9	Generic errors
<code>raise ValueError</code>	9	Invalid input
<code>return {}</code>	5	Empty result
<code>return []</code>	4	Empty list

6.3 Issues

1. **Silent Failures:** `return None` doesn't indicate what went wrong

2. **Generic Exceptions:** `RuntimeError` doesn't provide specific error type
3. **Inconsistent Null Objects:** Sometimes `None`, sometimes `{}`, sometimes `[]`

6.4 Recommended Pattern

Use Result Types or Specific Exceptions:

```
from typing import Optional, Union
from dataclasses import dataclass

@dataclass
class Result:
    """Result type for operations that may fail."""
    success: bool
    value: Optional[Any] = None
    error: Optional[str] = None

# Option 1: Result type (Rust-style)
def get_workpad(workpad_id: str) -> Result[Workpad]:
    """Get workpad by ID.

    Returns:
        Result with workpad if found, error message otherwise.
    """
    try:
        workpad = self._load_workpad(workpad_id)
        return Result(success=True, value=workpad)
    except WorkpadNotFoundError as e:
        return Result(success=False, error=str(e))

# Option 2: Specific exceptions (Python-style)
def get_workpad(workpad_id: str) -> Workpad:
    """Get workpad by ID.

    Returns:
        Workpad object

    Raises:
        WorkpadNotFoundError: If workpad doesn't exist
        WorkpadAccessError: If workpad is inaccessible
    """
    if not self._workpad_exists(workpad_id):
        raise WorkpadNotFoundError(f"Workpad {workpad_id} not found")

    return self._load_workpad(workpad_id)
```

6.5 Action Items

1. **Define custom exception hierarchy** (already exists partially)
2. **Audit all `return None` cases** - convert to exceptions or Result
3. **Replace generic `RuntimeError`** with specific exceptions
4. **Document error handling patterns** in contributing guide

Estimated Time: 4-6 hours

7. Type Hint Coverage

7.1 Summary

Coverage: 88.3% (490/555 public functions)

Gap: 65 functions without return type hints

Priority: Medium

7.2 Analysis

Good coverage overall, but missing types reduce IDE support and type safety.

7.3 Missing Type Hints by Module

Module	Missing	Total	Coverage
cli/	35	98	64%
ui/	18	145	88%
orchestration/	8	102	92%
engines/	4	82	95%

7.4 Recommendation

Add type hints to all public functions:

Before:

```
def create_workpad(title):
    """Create a new workpad."""
    ...
```

After:

```
def create_workpad(title: str) -> Workpad:
    """Create a new workpad.

    Args:
        title: Human-readable workpad title

    Returns:
        Newly created Workpad object
    """
    ...
```

7.5 Action Items

1. **Phase 1:** Add type hints to CLI commands (35 functions)
2. **Phase 2:** Add type hints to UI components (18 functions)
3. **Phase 3:** Add type hints to remaining modules (12 functions)
4. **Phase 4:** Run `mypy --strict` and fix any issues

Estimated Time: 3-4 hours

Tooling: Use `MonkeyType` to auto-generate hints based on runtime usage

8. Test Coverage Gaps

8.1 Summary

Total Modules: 46

Modules Without Dedicated Tests: 24 (52%)

Priority: High

8.2 Critical Gaps (Core Functionality)

Configuration Module

-  `config/manager.py` - No dedicated tests
-  `config/templates.py` - No dedicated tests

Impact: Configuration bugs can break entire system

Priority:  **Critical**

State Management

-  `state/git_sync.py` - No dedicated tests
-  `state/manager.py` - Some tests, incomplete
-  `state/schema.py` - No dedicated tests

Impact: State corruption can lose user data

Priority:  **Critical**

CLI Commands

-  `cli/ci_commands.py` - No tests
-  `cli/enhanced_commands.py` - No tests
-  `cli/commands.py` - Partial testing only

Impact: Broken CLI = broken user experience

Priority:  **Critical**

AI Provider Adapters

-  `orchestration/providers/abacus_adapter.py` - No dedicated tests
-  `orchestration/providers/openai_adapter.py` - No dedicated tests
-  `orchestration/providers/anthropic_adapter.py` - No dedicated tests

Impact: AI failures not caught until runtime

Priority:  **High**

8.3 UI Components (Lower Priority)

-  `ui/autocomplete.py`
-  `ui/command_palette.py`
-  `ui/enhanced_tui.py`
-  `ui/file_tree.py`
- (10 more UI modules)

Impact: UI bugs are visible but less critical

Priority: ⚠️ **Medium**

8.4 Test Creation Plan

Phase 1: Critical Modules (Week 1)

1. **Configuration Tests** (~4 hours)
 - Test config loading from YAML
 - Test environment variable overrides
 - Test profile switching
 - Test validation and defaults
2. **State Management Tests** (~6 hours)
 - Test state persistence
 - Test state recovery from corruption
 - Test Git ↔ State synchronization
 - Test concurrent access
3. **CLI Command Tests** (~8 hours)
 - Use Click's `CliRunner` for integration tests
 - Test all command variations
 - Test error handling
 - Test output formats (JSON, CSV, human-readable)
4. **AI Provider Tests** (~4 hours)
 - Mock API calls
 - Test error handling (network, auth, rate limits)
 - Test fallback logic
 - Test response parsing

Total Phase 1: ~22 hours (~3 days)

Phase 2: Medium Priority (Week 2)

- UI component tests
- Additional edge cases
- Performance tests

Total Phase 2: ~16 hours (~2 days)

8.5 Testing Strategy

CLI Testing Example:

```
# tests/test_cli_commands.py
from click.testing import CliRunner
from sologit.cli.commands import pad_create

def test_pad_create_success():
    """Test successful workpad creation."""
    runner = CliRunner()
    result = runner.invoke(pad_create, ["my-feature"])

    assert result.exit_code == 0
    assert "Created workpad" in result.output

def test_pad_create_invalid_name():
    """Test workpad creation with invalid name."""
    runner = CliRunner()
    result = runner.invoke(pad_create, ["invalid/name"])

    assert result.exit_code == 1
    assert "Error" in result.output
```

9. Documented-but-Missing Features

9.1 Summary

Features documented in specifications but not fully implemented.

9.2 Missing Features (From Coverage Matrix)

1. GitHub Actions Integration

Documented: FEATURES.md, ARCHITECTURE.md

Implementation: ❌ Not implemented

Impact: Users can't use GitHub Actions for CI

Required Work:

- Implement `workflows/ci_orchestrator.py::trigger_github_action()`
- Add GitHub Actions YAML templates
- Test with actual GitHub repository

Estimated Time: 4-6 hours

2. Security Scanning Hooks

Documented: FEATURES.md

Implementation: ❌ Not implemented

Impact: No automated security scanning

Required Work:

- Integrate with `trivy` or `bandit`
- Add to promotion gate checks
- Add configuration options

Estimated Time: 3-4 hours

3. Deployment Simulation

Documented: FEATURES.md

Implementation: ❌ Not implemented

Impact: Can't test deployments before promoting

Required Work:

- Define deployment simulation interface
- Implement basic simulation (script execution)
- Add to CI pipeline

Estimated Time: 4-5 hours

4. Notification System

Documented: ARCHITECTURE.md, FEATURES.md

Implementation: ❌ Not implemented

Impact: No alerts for rollbacks, budget limits, etc.

Required Work:

- Design notification interface
- Implement email/Slack/webhook backends
- Add notification triggers

Estimated Time: 6-8 hours

5. Credential Encryption

Documented: FEATURES.md (secure credential storage)

Implementation: ❌ Not implemented

Impact: API keys stored in plain text

Required Work:

- Implement encryption using `cryptography` library
- Add key derivation from user password
- Migrate existing configs

Estimated Time: 4-6 hours

Note: Deferred to Phase 3 (security hardening)

6. Command Aliases

Documented: FEATURES.md

Implementation: ❌ Not implemented

Impact: Users can't use shortcuts like `wp` for `workpad`

Required Work:

- Add alias configuration in `config.yaml`

- Implement alias resolution in CLI
- Add shell completion for aliases

Estimated Time: 2-3 hours

7. GUI Write Operations (Partial)

Documented: ACTION_PLAN.pdf

Implementation: ⚠️ Partial (mostly read-only)

Impact: Users can't create workpads from GUI

Required Work:

- Implement Tauri commands for create/delete/promote
- Add React UI components
- Add error handling and validation

Estimated Time: 6-8 hours

9.3 Total Missing Features

Count: 7 features

Total Effort: ~29-40 hours (~4-5 days)

Recommendation: Prioritize based on user impact

Priority Order:

1. 🔴 **Notification System** (high user impact)
 2. 🔴 **Credential Encryption** (security)
 3. 🟡 **GitHub Actions Integration** (medium user impact)
 4. 🟡 **GUI Write Operations** (nice-to-have)
 5. 🟡 **Command Aliases** (convenience)
 6. 🟢 **Security Scanning** (enhancement)
 7. 🟢 **Deployment Simulation** (advanced feature)
-

10. Commented-Out Code

10.1 Summary

Total Files with Significant Commented Code: 1

Priority: Low (minimal dead code)

10.2 Detailed Analysis

File: `sologit/engines/git_engine.py`

Commented Lines: 4

Status: ✅ **No action required** - Minimal debt

11. Overall Assessment

11.1 Codebase Health: ● Good

Category	Score	Status
Documentation	70%	● Good, needs API docs
Test Coverage	76%	● Good, critical gaps exist
Code Quality	75%	● Good, refactoring needed
Type Safety	88%	● Excellent
Technical Debt	5 TODOs	● Excellent
Dead Code	Minimal	● Excellent
Consistency	70%	● Needs improvement

11.2 Priority Matrix

● Critical (Do First)

1. Add tests for critical modules (config, state, CLI)
2. Refactor large functions (>200 lines)
3. Implement missing critical features (notifications, encryption)

● Important (Do Soon)

1. Consolidate duplicate code
2. Standardize naming conventions
3. Add type hints to remaining functions
4. Document undocumented public APIs

● Nice-to-Have (Do Later)

1. GUI write operations
2. Command aliases
3. Advanced features (security scanning, deployment simulation)

12. Recommendations for Phase 2 Refactor

12.1 Safe Refactoring Targets (No Behavior Change)

✓ These changes don't affect external behavior:

1. **Extract large functions** into smaller, testable units
2. **Consolidate duplicate helper functions**
3. **Rename functions** for consistency (internal APIs only)
4. **Add type hints** to existing functions
5. **Add docstrings** to public APIs

6. **Create base classes** for common TUI actions
7. **Organize utility functions** into `utils` modules

12.2 Changes Requiring Tests

⚠ **These need test coverage before refactoring:**

1. Configuration loading and validation
2. State persistence and recovery
3. CLI command execution
4. AI provider adapters

12.3 Changes Deferred (Behavior Change)

● **These would change external behavior - defer to later:**

1. Credential encryption (new feature)
2. Notification system (new feature)
3. GitHub Actions integration (new feature)
4. Security scanning hooks (new feature)

13. Action Plan Summary

Phase 2A: Test Coverage (Week 1)

- ☐ Add config manager tests
- ☐ Add state management tests
- ☐ Add CLI command tests
- ☐ Add AI provider adapter tests

Estimated Time: ~22 hours (~3 days)

Phase 2B: Code Refactoring (Week 2)

- ☐ Extract large functions (`execute_pair_loop`, `test_run`, `generate_commit_message`, `auto_merge.execute`)
- ☐ Consolidate duplicate utilities (`abort_with_error`, TUI actions)
- ☐ Standardize naming conventions
- ☐ Add missing type hints
- ☐ Add missing docstrings

Estimated Time: ~30 hours (~4 days)

Phase 2C: Documentation (Week 3)

- ☐ Update API documentation
- ☐ Create naming convention guide
- ☐ Document error handling patterns
- ☐ Update architecture diagrams

Estimated Time: ~8 hours (~1 day)

Total Phase 2 Effort

Estimated: 60 hours (~7-8 days)

14. Conclusion

The Solo-Git codebase is in **good health** with:

-  Minimal technical debt
-  Strong core implementation
-  Good overall test coverage (76%)

Key improvements needed:

1.  Test critical modules (config, state, CLI)
2.  Refactor large functions for maintainability
3.  Improve consistency (naming, error handling)
4.  Complete documentation

Next Steps: Proceed with Phase 2 Refactor Plan (see next document).

Change Log

- **2025-11-03:** Initial gap analysis completed
 - **2025-11-03:** Automated code scanning performed
 - **2025-11-03:** Manual review and prioritization added
-

This gap analysis will be updated as refactoring progresses.